⚠ PROFESSOR VERSION — COMPLETE SOLUTIONS ⚠ NOT FOR DISTRIBUTION

MUST University

Department of Computer Science

CS 141 – Data Structures & Algorithms

Spring 2026 • 1st Year – Software Engineering License

Professor Answer Key & Solution Guide

Chapter 3 – Linked Lists

Book Library — Singly Linked List

Contents

Prepared by the CS Department • Spring 2026

1. Book Node — Structure & Base Functions

The following code defines the Book node, the create function, the print function, and the free-all helper. Students must include all of this in every program.

1.1 Full Base Code (required header for all exercises)

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

// ── Book node ──────────────────────────────────────────────
struct Book {
    char   title[100];
    char   author[50];
    int    year;
    float  price;
    struct Book* next;
};
typedef struct Book Book;

// ── Create a new book node ─────────────────────────────────
Book* createBook(char* title, char* author, int year, float price) {
    Book* b = (Book*) malloc(sizeof(Book));
    if (b == NULL) { printf("Allocation failed!\n"); return NULL; }
    strcpy(b->title,  title);
    strcpy(b->author, author);
    b->year  = year;
    b->price = price;
    b->next  = NULL;
    return b;
```

```
}

// ── Print entire list ──────────────────────────────────────
void printBookList(Book* head) {
    Book* cur = head;
    int i = 1;
    while (cur != NULL) {
        printf("[%d] \"%s\" by %s (%d) — $%.2f\n",
               i, cur->title, cur->author, cur->year, cur->price);
        cur = cur->next;
        i++;
    }
    printf("--- NULL ---\n");
}

// ── Free every node in the list ────────────────────────────
void freeBookList(Book* head) {
    Book* temp;
    while (head != NULL) {
        temp = head;
        head = head->next;
        free(temp);
    }
}
```

Exercise 1 — Inserting Books into the Linked List

Objective: Implement the three core insertion functions for the Book Linked List.

Part A — Insert at the Beginning (O(1))

➡ Task 1.1 — insertAtBeginning()

Write: Book* insertAtBeginning(Book* head, char* title, char* author, int year, float price)

- Allocate a new Book node using createBook().
- Set newBook->next = head.
- Return newBook as the updated head.
- Handle the case where head is NULL (empty list).

☑ SOLUTION — insertAtBeginning()

```
Book* insertAtBeginning(Book* head,
                        char* title, char* author,
                        int year, float price) {
    Book* newBook = createBook(title, author, year, price);
    if (newBook == NULL) return head;   // allocation failed

    newBook->next = head;   // new node points to old head
    return newBook;          // new node becomes the new head
}

// ── Teaching note ──────────────────────────────────────────
// Works even when head == NULL: newBook->next = NULL → valid empty list
// O(1): no traversal needed at all
```

Expected output:

```
// head = insertAtBeginning(NULL, "You Don't Know JS", "Kyle Simpson", 2015, 34.99);
// head = insertAtBeginning(head, "Clean Code", "Robert C. Martin", 2008, 39.99);
[1] "Clean Code" by Robert C. Martin (2008) — $39.99
[2] "You Don't Know JS" by Kyle Simpson (2015) — $34.99
--- NULL ---
```

Part B — Insert at the End (O(n))

➡ Task 1.2 — insertAtEnd()

Write: Book* insertAtEnd(Book* head, char* title, char* author, int year, float price)

- Allocate a new Book node using createBook().
- If head == NULL, return the new node directly.
- Traverse until current->next == NULL.
- Link the last node to the new book node.
- Return the unchanged head.

☑ SOLUTION — insertAtEnd()

```
Book* insertAtEnd(Book* head,
                  char* title, char* author,
                  int year, float price) {
    Book* newBook = createBook(title, author, year, price);
    if (newBook == NULL) return head;

    if (head == NULL) return newBook;   // list was empty

    Book* current = head;
    while (current->next != NULL) {     // walk to last node
        current = current->next;
    }
    current->next = newBook;   // link last node to new node
    return head;               // head unchanged
}

// ── Teaching note ──────────────────────────────────────────
// The loop stops when current->next == NULL (current IS the last node)
// Do NOT write current != NULL — you would lose the reference
```

Expected output:

```
[1] "Clean Code" by Robert C. Martin (2008) — $39.99
[2] "The Pragmatic Programmer" by David Thomas (1999) — $45.00
[3] "Introduction to Algorithms" by Thomas H. Cormen (2009) — $89.95
--- NULL ---
```

Part C — Insert at a Given Position (O(n))

➡ Task 1.3 — insertAtPosition()

Write: Book* insertAtPosition(Book* head, char* title, char* author, int year, float price, int pos)

- If pos <= 1, delegate to insertAtBeginning().
- Walk the list with a counter to reach node at index pos−1.
- If out of range, print error, free the unused node, return head.
- Set newBook->next = current->next (point to successor).
- Set current->next = newBook (link predecessor to new node).

- Return the unchanged head.

☑ SOLUTION — insertAtPosition()

```
Book* insertAtPosition(Book* head,
                       char* title, char* author,
                       int year, float price, int pos) {
    // Special case: insert at front
    if (pos <= 1) return insertAtBeginning(head, title, author, year, price);

    Book* newBook = createBook(title, author, year, price);
    if (newBook == NULL) return head;

    Book* current = head;
    int   i = 1;
    // Walk to node at position (pos - 1)
    while (current != NULL && i < pos - 1) {
        current = current->next;
        i++;
    }

    if (current == NULL) {   // position out of range
        printf("Position %d is out of range.\n", pos);
        free(newBook);
        return head;
    }

    newBook->next  = current->next;  // new node -> successor
    current->next  = newBook;        // predecessor -> new node
    return head;
}

// ── Teaching note ──────────────────────────────────────────
// Order matters: set newBook->next BEFORE overwriting current->next
// Reversing these two lines loses the reference to the successor
```

Expected output — inserting at position 2:

```
// Initial:  Clean Code -> Introduction to Algorithms -> Design Patterns
// head = insertAtPosition(head, "The Pragmatic Programmer", "David Thomas", 1999, 45.00,
2);
[1] "Clean Code" by Robert C. Martin (2008) — $39.99
[2] "The Pragmatic Programmer" by David Thomas (1999) — $45.00
[3] "Introduction to Algorithms" by Thomas H. Cormen (2009) — $89.95
[4] "Design Patterns" by Erich Gamma (1994) — $54.99
--- NULL ---
```

Part D — Complete main() Program (All 6 Books)

➖ Task 1.4 — Build and Verify the Full List

Build a list of all 6 catalog books using all three insertion functions. Print after each insertion.

- insertAtEnd: Clean Code, Introduction to Algorithms, Design Patterns.
- insertAtBeginning: You Don't Know JS (appears first).
- insertAtPosition: The Pragmatic Programmer at position 2.
- insertAtEnd: The Art of Computer Programming at the end.
- Call freeBookList() at the end of main().

☑ SOLUTION — main() — Exercise 1

```
int main() {
    Book* head = NULL;

    // Step 1 — build backbone with insertAtEnd
    head = insertAtEnd(head, "Clean Code", "Robert C. Martin", 2008, 39.99f);
    head = insertAtEnd(head, "Introduction to Algorithms", "Thomas H. Cormen", 2009,
89.95f);
    head = insertAtEnd(head, "Design Patterns", "Erich Gamma", 1994, 54.99f);
    printf("After 3 insertAtEnd:\n"); printBookList(head);

    // Step 2 — prepend with insertAtBeginning
    head = insertAtBeginning(head, "You Don't Know JS", "Kyle Simpson", 2015, 34.99f);
    printf("\nAfter insertAtBeginning:\n"); printBookList(head);

    // Step 3 — insert The Pragmatic Programmer at position 2
    head = insertAtPosition(head, "The Pragmatic Programmer",
                            "David Thomas", 1999, 45.00f, 2);
    printf("\nAfter insertAtPosition (pos 2):\n"); printBookList(head);

    // Step 4 — append The Art of Computer Programming
    head = insertAtEnd(head, "The Art of Computer Programming",
                       "Donald Knuth", 1968, 120.00f);
    printf("\nFinal list (6 books):\n"); printBookList(head);

    // Cleanup
    freeBookList(head);
    return 0;
}
```

Final expected output:

```
[1] "You Don't Know JS" by Kyle Simpson (2015) — $34.99
[2] "The Pragmatic Programmer" by David Thomas (1999) — $45.00
[3] "Clean Code" by Robert C. Martin (2008) — $39.99
[4] "Introduction to Algorithms" by Thomas H. Cormen (2009) — $89.95
[5] "Design Patterns" by Erich Gamma (1994) — $54.99
[6] "The Art of Computer Programming" by Donald Knuth (1968) — $120.00
--- NULL ---
```

Hints & Reminders

→ Common student mistake: writing current->next = newBook BEFORE newBook->next = current->next — this loses the rest of the list.

→ Another common mistake: using = instead of strcpy() for char arrays — this copies a pointer, not the string.

→ For insertAtPosition with pos=1: always delegate to insertAtBeginning rather than trying to handle it inline.

→ Grading tip: test with an empty list (head=NULL) for all three functions — many students forget this edge case.

Exercise 2 — Deleting Books from the Linked List

Objective: Implement three deletion functions. Correctly re-link predecessor pointers and release memory for every deleted node.

Part A — Delete from the Beginning (O(1))

Task 2.1 — deleteFromBeginning()

Write: Book* deleteFromBeginning(Book* head)

- If head == NULL, print "List is empty." and return NULL.
- Save head in a temp pointer.
- Advance head to head->next.
- Print the title of the deleted book, then free(temp).
- Return the updated head.

SOLUTION — deleteFromBeginning()

```
Book* deleteFromBeginning(Book* head) {
    if (head == NULL) {
        printf("List is empty.\n");
        return NULL;
    }
    Book* temp = head;          // save reference to old head
    head = head->next;          // move head to second node
    printf("Deleted: \"%s\"\n", temp->title);
    free(temp);                 // release old head
    return head;
}

// ── Teaching note ──────────────────────────────────────────
// Must save temp BEFORE advancing head — otherwise we lose the pointer
// If list has exactly 1 node: head->next == NULL → head = NULL (correct)
```

Expected output:

```
// Before:  Clean Code -> The Pragmatic Programmer -> Introduction to Algorithms
// head = deleteFromBeginning(head);
Deleted: "Clean Code"
[1] "The Pragmatic Programmer" by David Thomas (1999) — $45.00
[2] "Introduction to Algorithms" by Thomas H. Cormen (2009) — $89.95
--- NULL ---
```

Part B — Delete from the End (O(n))

Task 2.2 — deleteFromEnd()

Write: Book* deleteFromEnd(Book* head)

- If head == NULL, print error and return NULL.
- If head->next == NULL (single node): print title, free it, return NULL.
- Traverse until current->next->next == NULL (second-to-last node).
- Print title of current->next, then free current->next.
- Set current->next = NULL and return head.

SOLUTION — deleteFromEnd()

```
Book* deleteFromEnd(Book* head) {
    if (head == NULL) {
        printf("List is empty.\n");
        return NULL;
    }
    // Special case: only one node
```

```
    if (head->next == NULL) {
        printf("Deleted: \"%s\"\n", head->title);
        free(head);
        return NULL;
    }
    // Walk to the second-to-last node
    Book* current = head;
    while (current->next->next != NULL) {
        current = current->next;
    }
    printf("Deleted: \"%s\"\n", current->next->title);
    free(current->next);   // free the last node
    current->next = NULL;  // current becomes the new last node
    return head;
}

// ── Teaching note ──────────────────────────────────────────
// Loop condition: current->next->next != NULL
//   stops when current->next IS the last node (its next == NULL)
// Alternative approach: use a 'prev' pointer initialized to NULL
//   and stop when current->next == NULL — both are valid
```

Expected output:

```
// Before:  The Pragmatic Programmer -> Introduction to Algorithms -> Design Patterns
// head = deleteFromEnd(head);
Deleted: "Design Patterns"
[1] "The Pragmatic Programmer" by David Thomas (1999) — $45.00
[2] "Introduction to Algorithms" by Thomas H. Cormen (2009) — $89.95
--- NULL ---
```

Part C — Delete by Title (Middle / Any Position) (O(n))

Task 2.3 — deleteByTitle()

Write: Book* deleteByTitle(Book* head, char* targetTitle)

- If head == NULL, return NULL.
- If head is the match (strcmp == 0): save, advance head, free, return new head.
- Otherwise traverse until current->next != NULL:
- If current->next matches: save it, relink current->next = saved->next, free saved.
- If not found: print "Book '[title]' not found in the list."

SOLUTION — deleteByTitle()

```
Book* deleteByTitle(Book* head, char* targetTitle) {
    if (head == NULL) {
        printf("List is empty.\n");
        return NULL;
    }

    // Case 1: target is the head node
    if (strcmp(head->title, targetTitle) == 0) {
        Book* temp = head;
        head = head->next;
        printf("Deleted: \"%s\"\n", temp->title);
        free(temp);
        return head;
    }

    // Case 2: search from second node onward
    Book* current = head;
```

```
    while (current->next != NULL) {
        if (strcmp(current->next->title, targetTitle) == 0) {
            Book* toDelete = current->next;
            current->next  = toDelete->next;  // bypass the node
            printf("Deleted: \"%s\"\n", toDelete->title);
            free(toDelete);
            return head;
        }
        current = current->next;
    }

    // Not found
    printf("Book '%s' not found in the list.\n", targetTitle);
    return head;
}

// ── Teaching note ──────────────────────────────────────────
// We check current->next (not current) so we always have the predecessor
// This avoids needing a separate 'prev' pointer
// strcmp returns 0 when strings are EQUAL — a very common confusion
```

Expected output — deleting a middle node:

```
// Before:  Clean Code -> The Pragmatic Programmer -> Introduction to Algorithms
// head = deleteByTitle(head, "The Pragmatic Programmer");
Deleted: "The Pragmatic Programmer"
[1] "Clean Code" by Robert C. Martin (2008) — $39.99
[2] "Introduction to Algorithms" by Thomas H. Cormen (2009) — $89.95
--- NULL ---
```

Part D — Complete main() with All Deletions

Task 2.4 — Full Deletion Sequence

Build the 6-book list from Exercise 1, then perform all three deletions in sequence. Print after each.

- Build the 6-book list using your Exercise 1 insertion functions.
- deleteFromBeginning() → removes You Don't Know JS (first).
- deleteByTitle("Design Patterns") → removes a middle book.
- deleteFromEnd() → removes The Art of Computer Programming (last).
- Free all remaining nodes with freeBookList().

SOLUTION — main() — Exercise 2

```
int main() {
    Book* head = NULL;

    // ── Build the 6-book list ──────────────────────────────
    head = insertAtEnd(head, "You Don't Know JS", "Kyle Simpson", 2015, 34.99f);
    head = insertAtEnd(head, "The Pragmatic Programmer", "David Thomas", 1999, 45.00f);
    head = insertAtEnd(head, "Clean Code", "Robert C. Martin", 2008, 39.99f);
    head = insertAtEnd(head, "Introduction to Algorithms", "Thomas H. Cormen", 2009,
89.95f);
    head = insertAtEnd(head, "Design Patterns", "Erich Gamma", 1994, 54.99f);
    head = insertAtEnd(head, "The Art of Computer Programming", "Donald Knuth", 1968,
120.00f);
    printf("Initial list:\n"); printBookList(head);

    // ── Deletion 1: from beginning ─────────────────────────
    head = deleteFromBeginning(head);
```

```
    printf("\nAfter deleteFromBeginning:\n"); printBookList(head);

    // ── Deletion 2: by title (middle node) ──────────────────────
    head = deleteByTitle(head, "Design Patterns");
    printf("\nAfter deleteByTitle(Design Patterns):\n"); printBookList(head);

    // ── Deletion 3: from end ────────────────────────────────────
    head = deleteFromEnd(head);
    printf("\nAfter deleteFromEnd:\n"); printBookList(head);

    // ── Cleanup ─────────────────────────────────────────────────
    freeBookList(head);
    return 0;
}
```

Full expected console trace:

```
Initial list:
[1] "You Don't Know JS" by Kyle Simpson (2015) — $34.99
[2] "The Pragmatic Programmer" by David Thomas (1999) — $45.00
[3] "Clean Code" by Robert C. Martin (2008) — $39.99
[4] "Introduction to Algorithms" by Thomas H. Cormen (2009) — $89.95
[5] "Design Patterns" by Erich Gamma (1994) — $54.99
[6] "The Art of Computer Programming" by Donald Knuth (1968) — $120.00
--- NULL ---

After deleteFromBeginning:
Deleted: "You Don't Know JS"
[1] "The Pragmatic Programmer" by David Thomas (1999) — $45.00
... (5 books)

After deleteByTitle(Design Patterns):
Deleted: "Design Patterns"
[1] "The Pragmatic Programmer" by David Thomas (1999) — $45.00
... (4 books)

After deleteFromEnd:
Deleted: "The Art of Computer Programming"
[1] "The Pragmatic Programmer" by David Thomas (1999) — $45.00
[2] "Clean Code" by Robert C. Martin (2008) — $39.99
[3] "Introduction to Algorithms" by Thomas H. Cormen (2009) — $89.95
--- NULL ---
```

Hints & Reminders

→ Grading tip: test deleteFromBeginning on a 1-node list — result must be NULL, not a dangling pointer.

→ Grading tip: test deleteByTitle with a non-existent title — "not found" message must appear, list unchanged.

→ Grading tip: test deleteFromEnd on an empty list and a single-node list — both edge cases required.

→ Common mistake: freeing the node BEFORE reading its title for the printed message.

→ Common mistake: in deleteByTitle, using = instead of strcmp() for string comparison.

Bonus Challenge — Solution Guide

Bonus A — deleteMostExpensive()

SOLUTION — deleteMostExpensive()

```
Book* deleteMostExpensive(Book* head) {
    if (head == NULL) { printf("List is empty.\n"); return NULL; }

    // Pass 1 — find the maximum price
    float maxPrice = head->price;
    Book* cur = head->next;
    while (cur != NULL) {
        if (cur->price > maxPrice) maxPrice = cur->price;
        cur = cur->next;
    }

    // Pass 2 — delete the first node whose price == maxPrice
    // Reuse deleteByTitle logic with a price check instead
    if (head->price == maxPrice) {
        Book* temp = head;
        head = head->next;
        printf("Deleted most expensive: \"%s\" ($%.2f)\n", temp->title, temp->price);
        free(temp);
        return head;
    }
    cur = head;
    while (cur->next != NULL) {
        if (cur->next->price == maxPrice) {
            Book* toDelete = cur->next;
            cur->next = toDelete->next;
            printf("Deleted most expensive: \"%s\" ($%.2f)\n",
                   toDelete->title, toDelete->price);
            free(toDelete);
            return head;
        }
        cur = cur->next;
    }
    return head;
}

// ── Teaching note ──────────────────────────────────────────
// Two-pass approach is cleaner than trying to track max + predecessor simultaneously
// Float comparison with == is acceptable here since we stored the exact value
```

Bonus B — deleteByYear()

SOLUTION — deleteByYear()

```
Book* deleteByYear(Book* head, int year) {
    // Remove matching head nodes first (may be multiple consecutive)
    while (head != NULL && head->year == year) {
        Book* temp = head;
        head = head->next;
        printf("Deleted: \"%s\" (%d)\n", temp->title, temp->year);
        free(temp);
    }
    if (head == NULL) return NULL;

    // Remove matching nodes in the rest of the list
    Book* current = head;
    while (current->next != NULL) {
        if (current->next->year == year) {
            Book* toDelete = current->next;
            current->next  = toDelete->next;
            printf("Deleted: \"%s\" (%d)\n", toDelete->title, toDelete->year);
            free(toDelete);
```

```
            // Do NOT advance current — next node may also match
        } else {
            current = current->next;
        }
    }
    return head;
}

// ── Teaching note ──────────────────────────────────────────
// Key: do NOT advance current after a deletion — the new current->next
//   might also match. Advancing would skip it.
// The while loop at the top handles consecutive matching head nodes cleanly
```

Bonus C — insertSorted()

☑ SOLUTION — insertSorted() — alphabetical by title

```
Book* insertSorted(Book* head,
                   char* title, char* author,
                   int year, float price) {
    Book* newBook = createBook(title, author, year, price);
    if (newBook == NULL) return head;

    // New book comes before the current head
    if (head == NULL || strcmp(title, head->title) <= 0) {
        newBook->next = head;
        return newBook;
    }

    // Find the correct insertion point
    Book* current = head;
    while (current->next != NULL &&
           strcmp(title, current->next->title) > 0) {
        current = current->next;
    }
    // Insert between current and current->next
    newBook->next  = current->next;
    current->next  = newBook;
    return head;
}

// ── Teaching note ──────────────────────────────────────────
// strcmp(a, b) < 0  means a comes BEFORE b alphabetically
// strcmp(a, b) > 0  means a comes AFTER  b alphabetically
// strcmp(a, b) == 0 means a and b are identical
// The <= 0 check at the head handles both 'before head' and 'same title'
```

Complete Compilable Reference Program (compile with: gcc -o books books.c)

The following is the full standalone C program combining all functions. Use it to verify student outputs or as a demo during class.

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

// ── Node ──────────────────────────────────────────────────
struct Book { char title[100]; char author[50]; int year; float price; struct Book*
next; };
typedef struct Book Book;
```

```
// ── Base functions ──────────────────────────────────────
Book* createBook(char* t, char* a, int y, float p) {
    Book* b=(Book*)malloc(sizeof(Book));
    if(!b){printf("Alloc failed\n");return NULL;}
    strcpy(b->title,t); strcpy(b->author,a); b->year=y; b->price=p; b->next=NULL;
    return b;
}
void printBookList(Book* h) {
    int i=1;
    while(h){printf("[%d] \"%s\" by %s (%d) — $%.2f\n",i,h->title,h->author,h->year,h-
>price);h=h->next;i++;}
    printf("--- NULL ---\n");
}
void freeBookList(Book* h){Book* t;while(h){t=h;h=h->next;free(t);}}

// ── Insertions ──────────────────────────────────────────
Book* insertAtBeginning(Book* h,char* t,char* a,int y,float p){
    Book* n=createBook(t,a,y,p); if(!n)return h; n->next=h; return n;
}
Book* insertAtEnd(Book* h,char* t,char* a,int y,float p){
    Book* n=createBook(t,a,y,p); if(!n)return h;
    if(!h)return n;
    Book* c=h; while(c->next)c=c->next; c->next=n; return h;
}
Book* insertAtPosition(Book* h,char* t,char* a,int y,float p,int pos){
    if(pos<=1)return insertAtBeginning(h,t,a,y,p);
    Book* n=createBook(t,a,y,p); if(!n)return h;
    Book* c=h; int i=1;
    while(c&&i<pos-1){c=c->next;i++;}
    if(!c){printf("Position %d out of range\n",pos);free(n);return h;}
    n->next=c->next; c->next=n; return h;
}

// ── Deletions ───────────────────────────────────────────
Book* deleteFromBeginning(Book* h){
    if(!h){printf("Empty list\n");return NULL;}
    Book* t=h; h=h->next;
    printf("Deleted: \"%s\"\n",t->title); free(t); return h;
}
Book* deleteFromEnd(Book* h){
    if(!h){printf("Empty list\n");return NULL;}
    if(!h->next){printf("Deleted: \"%s\"\n",h->title);free(h);return NULL;}
    Book* c=h; while(c->next->next)c=c->next;
    printf("Deleted: \"%s\"\n",c->next->title);
    free(c->next); c->next=NULL; return h;
}
Book* deleteByTitle(Book* h,char* target){
    if(!h){printf("Empty list\n");return NULL;}
    if(strcmp(h->title,target)==0){
        Book* t=h;h=h->next;printf("Deleted: \"%s\"\n",t->title);free(t);return h;
    }
    Book* c=h;
    while(c->next){
        if(strcmp(c->next->title,target)==0){
            Book* d=c->next;c->next=d->next;
            printf("Deleted: \"%s\"\n",d->title);free(d);return h;
        }
        c=c->next;
    }
    printf("'%s' not found\n",target); return h;
}

// ── main ────────────────────────────────────────────────
int main(){
    Book* head=NULL;
    head=insertAtEnd(head,"You Don't Know JS","Kyle Simpson",2015,34.99f);
```

```
    head=insertAtEnd(head,"The Pragmatic Programmer","David Thomas",1999,45.00f);
    head=insertAtEnd(head,"Clean Code","Robert C. Martin",2008,39.99f);
    head=insertAtEnd(head,"Introduction to Algorithms","Thomas H. Cormen",2009,89.95f);
    head=insertAtEnd(head,"Design Patterns","Erich Gamma",1994,54.99f);
    head=insertAtEnd(head,"The Art of Computer Programming","Donald Knuth",1968,120.00f);
    printf("=== Initial list ===\n"); printBookList(head);
    head=deleteFromBeginning(head);
    printf("\n=== After deleteFromBeginning ===\n"); printBookList(head);
    head=deleteByTitle(head,"Design Patterns");
    printf("\n=== After deleteByTitle ===\n"); printBookList(head);
    head=deleteFromEnd(head);
    printf("\n=== After deleteFromEnd ===\n"); printBookList(head);
    freeBookList(head);
    return 0;
}
```

Summary Operation Reference

Operation	Time	Key Action
`insertAtBeginning()`	O(1)	newBook->next = head; return newBook
`insertAtEnd()`	O(n)	Walk to last; last->next = newBook; return head
`insertAtPosition()`	O(n)	Walk to pos−1; newBook->next=cur->next; cur->next=newBook
`deleteFromBeginning()`	O(1)	temp=head; head=head->next; free(temp); return head
`deleteFromEnd()`	O(n)	Walk to 2nd-to-last; free last; cur->next=NULL
`deleteByTitle()`	O(n)	strcmp match; cur->next=toDelete->next; free(toDelete)
`deleteMostExpensive()`	O(n)	2-pass: find maxPrice, then delete first match
`deleteByYear()`	O(n)	Loop all nodes; do NOT advance after deletion
`insertSorted()`	O(n)	strcmp walk; insert at first pos where next > title

Grading Rubric — Suggested Point Distribution

Function	Correct logic	Edge cases	Memory	Total
`insertAtBeginning()`	5	2 (empty list)	1 (free on fail)	8
`insertAtEnd()`	5	2 (empty list)	1	8
`insertAtPosition()`	6	3 (pos≤1, out-range)	1	10
`deleteFromBeginning()`	5	2 (empty, 1 node)	1	8
`deleteFromEnd()`	5	2 (empty, 1 node)	1	8
`deleteByTitle()`	6	3 (head, not found)	1	10
`freeBookList()` + `main()`	5	2 (all insertions mix)	1	8
TOTAL	**37**	**16**	**7**	**60**